

Introduction

Binary tree problems show up in almost every coding interview loop, and the ones that trip people up aren't the ones that require a clever trick — they're the ones that punish redundant work. LeetCode 2265, "Count Nodes Equal to Average of Subtree," looks harmless on the surface: for each node, check whether its value equals the average of its own subtree. But the naive way to check "each node" ends up recomputing the same sums over and over, and that's exactly the kind of inefficiency interviewers are trained to spot.

What makes this problem worth your time isn't the final answer — it's the pattern underneath it. Passing information *up* a tree instead of *down* into it is one of the most reusable ideas in tree algorithms, and it shows up again in problems about diameter, balance, and subtree sums. Once this pattern clicks, a whole category of "subtree property" questions stops being scary.

Problem Overview

You're given the root of a binary tree. Every node holds an integer value. For a given node, its **subtree** is that node plus everything below it — left child, right child, and their descendants, all the way down.

For each node, compute the average of every value in its subtree, using **integer (floor) division** — so 11 divided by 2 is 5, not 5.5. If that average equals the node's own value, the node counts.

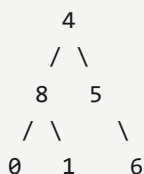
Your job: return the total number of nodes across the whole tree where this condition holds.

Two details matter more than they look:

- The average is computed over the node's *entire* subtree, not just its immediate children.
- Rounding is always down. A node whose true average is 4.9 does **not** match a value of 5.

Example

Take this tree:



Work from the bottom up:

- **Node 0** — leaf. Subtree is just $\{0\}$. Average = $0 / 1 = 0$. Matches ($0 == 0$). ✓
- **Node 1** — leaf. Subtree is $\{1\}$. Average = 1. Matches. ✓
- **Node 6** — leaf. Subtree is $\{6\}$. Average = 6. Matches. ✓
- **Node 8** — subtree is $\{8, 0, 1\}$. Sum = 9, count = 3, average = 3. $8 \neq 3$. ✗
- **Node 5** — subtree is $\{5, 6\}$. Sum = 11, count = 2, average = 5 (floor of 5.5). Matches. ✓
- **Node 4 (root)** — subtree is the whole tree: $\{4, 8, 5, 0, 1, 6\}$. Sum = 24, count = 6, average = 4. Matches. ✓

Total matches: **5** — nodes 0, 1, 6, 5, and 4. That's the answer.

Notice node 8 is the only miss, and it's a good example of why you can't judge a node by its children's values alone — you have to look at the whole subtree beneath it.

Intuition

The instinctive first move is: for every node, walk its subtree, add everything up, count the nodes, divide, compare. That works. It's also wasteful, because a node's subtree is nothing more than its left child's subtree, plus its right child's subtree, plus itself.

Think of it like a company org chart. If you want to know the total salary under a manager, you don't re-interview every employee in that manager's chain every time you check a different manager. Each manager already gets a rolled-up number from their direct reports and just adds their own salary on top. The rollup happens once per person, not once per person *per manager who asks*.

That's the shift: instead of asking "what's under me?" by walking down every time, have each node hand its total *up* to whoever asks for it, once. A postorder traversal — visit left, visit right, then process the current node — is the natural way to do this, because by the time you're standing at a node, both of its children have already reported their numbers.

And here's the part that makes it elegant: you don't need a separate pass to "check every node" after computing totals. The check can happen at the exact moment a node receives its children's totals, inside the same traversal that computes them. One pass does both jobs.

Brute Force Solution

Idea: For every node in the tree, run a full traversal of its subtree to compute the sum and count, then compare the average to the node's value.

Algorithm: 1. Traverse the tree (any order — preorder is fine). 2. At each node, run a second traversal starting from that node to sum its subtree and count its nodes. 3. Compare $\text{floor}(\text{sum} / \text{count})$ to the node's value; increment a counter on match.

Advantages: Straightforward to write, easy to reason about, no shared state to get confused by.

Disadvantages: Massive redundant work. A node near the root triggers a full subtree walk that revisits nodes that were *just* walked as part of a child's subtree computation moments earlier.

Complexity: For each of the n nodes, the subtree walk can touch up to n nodes in the worst case (a skewed tree), giving **$O(n^2)$ time**. Space is $O(h)$ for recursion, where h is tree height.

Given the problem's constraint of at most 1,000 nodes, this would technically pass — but it doesn't scale, and it's not the answer an interviewer wants to see once you understand the redundancy.

Optimal Solution

Switch to a bottom-up postorder DFS where each call returns two numbers: the **sum** and the **node count** of the subtree rooted at that call.

Step by step:

1. **Base case:** if the current node is `None`, return `(0, 0)` — an empty branch contributes nothing to sum or count, so it never distorts a parent's totals.
2. **Recurse left, recurse right:** get `(left_sum, left_count)` and `(right_sum, right_count)` from the two children. This is the postorder step — children are resolved before the parent uses their results.
3. **Combine:** this node's subtree sum is `left_sum + right_sum + node.val`; this node's subtree count is `left_count + right_count + 1`.
4. **Check:** if `total_sum // total_count == node.val`, increment the running counter.
5. **Return:** hand `(total_sum, total_count)` up to the caller (the parent), so the parent can do the same thing.

Node Left (sum, count) Right (sum, count) Own sum Own count Average Match?

0	(0,0)	(0,0)	0	1	0	✓
1	(0,0)	(0,0)	1	1	1	✓
8	(0,1)	(1,1)	9	3	3	✗
6	(0,0)	(0,0)	6	1	6	✓
5	(0,0)	(6,1)	11	2	5	✓
4	(9,3)	(11,2)	24	6	4	✓

Every node is visited exactly once, and every subtree's totals are computed exactly once — no rework, no second pass.

Python Code

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def averageOfSubtree(self, root: TreeNode) -> int:
        self.count = 0

        def dfs(node):
            if not node:
                return 0, 0 # (sum, count) for an empty subtree

            left_sum, left_count = dfs(node.left)
            right_sum, right_count = dfs(node.right)

            total_sum = left_sum + right_sum + node.val
            total_count = left_count + right_count + 1

            if total_sum // total_count == node.val:
                self.count += 1

            return total_sum, total_count

        dfs(root)
        return self.count
```

Code Walkthrough

- `self.count = 0` — a running tally shared across every recursive call, since `dfs` itself only returns subtree totals, not the answer.
- `def dfs(node):` — the recursive helper. It always returns a `(sum, count)` tuple for the subtree rooted at `node`.
- `if not node: return 0, 0` — the base case for a missing child. Returning zeros means a leaf's single child (or a leaf itself) never gets skewed by a phantom branch.
- `left_sum, left_count = dfs(node.left)` and the line after it — this is the postorder order of operations: resolve both children *before* touching the current node's own totals.

- `total_sum = left_sum + right_sum + node.val / total_count = left_count + right_count + 1` — the actual rollup: this node's own value and its own presence get folded into what its children already reported.
- `if total_sum // total_count == node.val:` — floor division (`//`), not `/`. This is the check the whole problem hinges on, and it has to happen after totals are combined, using totals that include the current node itself.
- `return total_sum, total_count` — hands the combined numbers up to whichever call invoked this one (the parent), continuing the rollup.
- `dfs(root)` at the end — triggers the whole walk; the return value is discarded because everything that matters was recorded in `self.count`.

Dry Run

Using the tree from the example (4 root, 8 and 5 children, 0 / 1 under 8 , 6 under 5):

1. `dfs(4)` calls `dfs(8)`.
2. `dfs(8)` calls `dfs(0)` → base case children return `(0,0)`, `(0,0)` → node 0: sum=0, count=1, `0//1=0==0` → count=1. Returns `(0,1)`.
3. `dfs(8)` calls `dfs(1)` → same logic → node 1: sum=1, count=1, match → count=2. Returns `(1,1)`.
4. Back in `dfs(8)`: sum = 0+1+8 = 9, count = 1+1+1 = 3, `9//3=3 ≠ 8` → no increment. Returns `(9,3)`.
5. `dfs(4)` calls `dfs(5)`.
6. `dfs(5)` calls `dfs(None)` (left) → `(0,0)`; calls `dfs(6)` → node 6: sum=6, count=1, match → count=3. Returns `(6,1)`.
7. Back in `dfs(5)`: sum = 0+6+5 = 11, count = 0+1+1 = 2, `11//2=5==5` → match → count=4. Returns `(11,2)`.
8. Back in `dfs(4)`: sum = 9+11+4 = 24, count = 3+2+1 = 6, `24//6=4==4` → match → count=5.
9. `dfs(4)` returns `(24,6)`; `averageOfSubtree` returns `self.count = 5`.

Matches the expected answer.

Complexity Analysis

Time: O(n). Each node is visited exactly once by `dfs`, and the work done at each node (adding two numbers, one comparison) is O(1). No node's subtree is ever re-walked, because totals are passed up rather than recomputed.

Space: $O(h)$. The only extra memory is the recursion call stack, which grows to the height of the tree, h . In the worst case (a completely skewed tree) $h = n$, giving $O(n)$; for a balanced tree, $h = O(\log n)$.

Compare that to the brute-force $O(n^2)$ — for the problem's constraint of 1,000 nodes, the difference is a million-ish operations versus a thousand.

Alternative Solutions

An iterative postorder traversal using an explicit stack achieves the same $O(n)$ time and avoids Python's recursion limit for very deep trees (Python's default recursion limit is around 1,000, which happens to sit right at this problem's node cap — worth knowing if you ever see a similar problem without that constraint). The tradeoff is more bookkeeping: you'd track child totals for each node manually with a dictionary or by pushing partial results onto the stack, which adds code without changing the asymptotic complexity. For interview purposes, the recursive version is almost always preferred unless you're explicitly asked to avoid recursion.

Edge Cases

- **Single-node tree:** the root is a leaf; its subtree is itself, so it always matches (average of one number is that number). Count = 1.
- **All nodes equal:** e.g., every value is 5 — every subtree average is 5, so every node matches. Count = n .
- **Skewed tree (linked-list shape):** still $O(n)$ time, but recursion depth hits n , which is where an iterative version becomes worth considering for very large inputs.
- **Minimum value nodes (value 0):** make sure your check doesn't special-case zero — `0 // count == 0` behaves like any other comparison.
- **Root doesn't match but a deep leaf does:** confirms the algorithm doesn't assume any correlation between a node's own value and whether it "should" match.

Common Mistakes

1. **Using float division (`/`) instead of floor division (`//`).** The problem explicitly rounds down; `/` will give you a float that never exactly equals an integer node value except by coincidence.
2. **Checking the average against children's totals only, forgetting to include the current node's own value and count.** This silently checks the wrong subtree.

3. **Returning sum and count as two separate values or instance attributes instead of one tuple.** This risks reading a stale count from before the current node was folded in, especially if the two updates aren't atomic within the call.
4. **Forgetting the base case returns `(0, 0)` and not `(node.val, 1)` or something nonzero.** A None child should never contribute to totals.
5. **Doing two separate passes** — one to compute all sums, one to check all nodes — when a single postorder pass already does both, doubling the traversal for no benefit.
6. **Off-by-one in the count**, forgetting to add 1 for the current node itself when combining children's counts.

Interview Questions

- How would you modify this to return the *node* with the closest average instead of just counting matches?
- What changes if the tree can have negative values? (Floor division behaves differently for negative numbers — worth testing.)
- Could you solve this iteratively without recursion? Walk through how you'd simulate the postorder stack.
- How would this pattern extend to a problem asking for the maximum subtree sum, or checking whether a tree is height-balanced?
- What's the space complexity if the tree is heavily skewed versus balanced, and why does that matter for very large inputs?

Similar Problems

- **LeetCode 104 (Maximum Depth of Binary Tree)** — same bottom-up recursive shape, simpler payload.
- **LeetCode 110 (Balanced Binary Tree)** — another postorder problem where each call returns info the parent needs to make its own decision.
- **LeetCode 543 (Diameter of Binary Tree)** — classic "return one thing, but track a separate global answer" pattern, just like this problem's `self.count`.
- **LeetCode 863 / 988 / 1120 (various subtree aggregate problems)** — all reuse the "return sum/count/height from children, combine at parent" idea.

Key Takeaways

The lesson isn't really about averages or binary trees specifically — it's about recognizing when a computation is naturally recursive on subproblems that overlap. Whenever a node's answer can be built entirely from its children's answers, resist the urge to recompute from scratch at every level. Return a bundled tuple, combine it at the parent, and check whatever condition you need in that same pass. This single pattern — postorder DFS with a bottom-up rollup — solves a wide swath of tree interview questions with a nearly identical skeleton.

Watch the Video

If you want to see this traced out on the whiteboard in real time, including the full dry run and the two guess-before-I-answer checkpoints, watch the full breakdown here: {LINK}

About the Series

This walkthrough is part of **Fun with Learning Technology**, a series built around making coding interview prep actually stick — not just showing a working solution, but building the intuition an experienced engineer uses to find it in the first place. Every episode pairs a real problem with the reasoning behind it, so the pattern transfers to problems you haven't seen yet.

Call To Action

If this helped the problem click, drop a like on the video and subscribe on YouTube so you don't miss the next one. For deeper written breakdowns like this one, subscribe to the Substack newsletter. And if you got stuck somewhere or solved it a different way, leave a comment — I read every one and it genuinely shapes what gets covered next.

Quick Review

When a tree problem asks about subtree averages/sums, what approach?

Bottom-up post-order recursion — compute sum and count per subtree once, avoid re-walking subtrees repeatedly.

Time complexity of counting nodes equal to their subtree average?

$O(n)$ — each node visited once during the single post-order pass.

Space complexity of the bottom-up subtree-average solution?

$O(h)$ for recursion stack (h = tree height), plus $O(1)$ extra per node.

Trickiest edge case / common bug in this problem?

Integer division truncation — average = sum/count must floor, not round; forgetting this misses valid matches.

Recomputing subtree sum per node (naive) vs bottom-up helper — why avoid the first?

Naive re-walks each subtree from every ancestor, giving $O(n^2)$ worst case on skewed trees; bottom-up is $O(n)$.

Interview follow-up: how would you return the actual matching subtree roots, not just a count?

Append node to a results list inside the same post-order helper when $\text{sum}/\text{count} == \text{node.val}$, instead of incrementing a counter.